Lab 6: Classes and Objects

Lab Overview

This lab introduces classes and objects in C++. You will learn how to model real-world entities as classes, bundle data and behaviour(functions) together, and control how objects are created and used. Understanding classes and objects is the foundation for all advanced C++ programming.

Learning Objectives

#	Learning Objective
1	Define a class with member data and member functions
2	Write member functions outside the class body using the scope resolution operator
3	Create objects and access their members using the dot operator
4	Pass objects as arguments to functions
5	Initializing the Objects: setter method and Constructors
6	Use structs as member data inside a class
7	Return objects from functions
8	Declare and use static data members shared across all objects
9	Write const member functions and create const objects
10	Data Hiding: Private and Public access (Encapsulation)

Submission Details

You are required to submit all tasks at GitHub Classroom. Create a separate file for each task and upload all files. File names must follow the pattern task_1.cpp, task_2.cpp, etc. Submit by April 6th.

B-Section: https://classroom.github.com/a/gSPHmaUL

C-Section: https://classroom.github.com/a/Rh-UrUL0

1. Defining a Class

A class is a user-defined type that bundles related data (member variables) and behaviour (member functions) into a single unit. Think of a class as a blueprint; an object is a concrete instance built from that blueprint.

1.1 Member Data and Member Functions

Member data (also called attributes or fields) store the state of an object. Member functions (also called methods) define what the object can do. Both live inside the class body.

```
class ClassName {
 public:
 data_type memberVariable;
 return_type memberFunction(parameters);
};
```

The keyword public makes members accessible from outside the class. The keyword private (the default) restricts access to within the class only.

Example Code

```
#include <iostream>
using namespace std;
class Car {
public:
    string brand;   // member data
    int year;       // member data
    void display() {   // member function
        cout << "Brand: " << brand << ", Year: " << year << endl;
    }
};
int main() {
    Car c;
    c.brand = "Toyota";
    c.year  = 2022;
    c.display();
    return 0;
}
```

Expected Output: Brand: Toyota, Year: 2022

Key Points

- A class definition ends with a semicolon after the closing brace.
- Member functions have access to all member data of the same object automatically.

- Separate data from behaviour: data describes what an object is; functions describe what it does.

2. Writing Member Functions Outside the Class

For larger programs it is common to declare member functions inside the class and define them outside. The scope resolution operator (::) tells the compiler which class the function belongs to.

```
return_type ClassName::functionName(parameters) { ... }
```

Example Code

```
#include <iostream>
using namespace std;

class Rectangle {
public:
    double length;
    double width;

    double area();       // declaration only
    double perimeter();  // declaration only
};

// Definitions outside the class
double Rectangle::area() {
    return length * width;
}

double Rectangle::perimeter() {
    return 2 * (length + width);
}

int main() {
    Rectangle r;
    r.length = 5.0;
    r.width  = 3.0;
    cout << "Area:      " << r.area()      << endl;
    cout << "Perimeter: " << r.perimeter() << endl;
    return 0;
}
```

Expected Output: Area: 15 | Perimeter: 16

Key Points

- The class body acts as an interface. It lists what functions exist.
- External definitions keep the class declaration short and readable.

Practice Tasks

Task 1: Define a class Circle with a member double radius. Declare the functions area() and circumference() inside the class but define them outside using ::. Call both from main() and print the results.

3. Creating Objects

An object is a variable of a class type. Declaring an object allocates memory for all its member data. You use the dot operator (.) to access members.

```
ClassName objectName;       //object
```

Example Code

```
#include <iostream>
using namespace std;

class Student {
public:
    string name;
    int    rollNo;
    double gpa;

    void printInfo() {
        cout << rollNo << " - " << name << " - GPA: " << gpa << endl;
    }
};

int main() {
    Student s1;       // create first object
    s1.name   = "Ali";
```

```
    s1.rollNo = 101;
    s1.gpa    = 3.8;

    Student s2;       // create second object
    s2.name   = "Sara";
    s2.rollNo = 102;
    s2.gpa    = 3.5;

    s1.printInfo();
    s2.printInfo();
    return 0;
}
```

Expected Output: 101 - Ali - GPA: 3.8 101 - Sara - GPA: 3.5

Key Points

- Each object gets its own copy of all member data.
- Member functions are shared; they act on whichever object calls them.
- You can create arrays of objects just like arrays of int or double.

Practice Tasks

Task 2: Create a class Book with members title (string), author (string), and price (double). Create an array of 3 Book objects, fill them with data, and print each book's details using a loop.

4. Objects as Function Arguments

Objects can be passed to regular (non-member) functions or to other member functions just like any built-in type. As with primitive types, you can pass by value (a copy) or by reference (the original).

4.1 Pass by Value

A full copy of the object is made. Changes inside the function do not affect the original.

```
#include <iostream>
```

```
using namespace std;

class Point {
public:
    int x, y;
};

void printPoint(Point p) {          // pass by value
    cout << "(" << p.x << ", " << p.y << ")" << endl;
}

int main() {
    Point p1;
    p1.x = 3;  p1.y = 7;
    printPoint(p1);
    return 0;
}
```

Expected Output: (3, 7)

4.2 Pass by Reference

The actual object is passed. The function can read or modify the original. Use const reference to read without copying and without allowing changes.

```
#include <iostream>
#include <cmath>
using namespace std;

class Point {
public:
    int x, y;
};

void translate(Point &p, int dx, int dy) {  // pass by reference
    p.x += dx;
    p.y += dy;
```

```
}

double distance(const Point &a, const Point &b) {
    int dx = a.x - b.x;
    int dy = a.y - b.y;
    return sqrt(dx*dx + dy*dy);
}

int main() {
    Point p1; p1.x = 0; p1.y = 0;
    Point p2; p2.x = 3; p2.y = 4;
    translate(p1, 1, 2);
    cout << "Translated: (" << p1.x << ", " << p1.y << ")" << endl;
    cout << "Distance: "   << distance(p1, p2)             << endl;
    return 0;
}
```

Expected Output: Translated: (1, 2) Distance: 2.83

Practice Tasks

Task 3: Write a function compare(Rectangle a, Rectangle b) that receives two Rectangle objects (with length and width) by value and prints which one has the larger area. Call it from main().

5. Initializing the Objects

A constructor is a special member function that runs automatically whenever an object is created. It is used to initialise member data to safe default values. Constructors have the same name as the class and no return type. For manual initialization, we use setter method.

5.1 Setter Method (Mutator)

This approach uses a setter functions to set member data. It separates validation from construction.

```
#include <iostream>
using namespace std;
```

```
class BankAccount {
private:
    string owner;
    double balance;
public:
    void setOwner(string o)   { owner   = o; }
    void setBalance(double b) { balance = (b >= 0) ? b : 0; }

    BankAccount(string o, double b) {  // constructor calls setters
        setOwner(o);
        setBalance(b);
    }

    void display() {
        cout << owner << " - Balance: " << balance << endl;
    }
};

int main() {
    BankAccount acc("Ahmed", 5000.0);
    acc.display();
    return 0;
}
```

Expected Output: Ahmed - Balance: 5000

5.2 Constructor with Same Name as Class

The standard form: the constructor shares the class name exactly, has no return type, and runs the moment an object is declared.

```
#include <iostream>
using namespace std;

class Temperature {
public:
    double celsius;
```

```
    Temperature(double c) {   // same name as class
        celsius = c;
    }

    double toFahrenheit() { return celsius * 9.0/5.0 + 32; }
};

int main() {
    Temperature t(100.0);
    cout << t.celsius << "C = " << t.toFahrenheit() << "F" << endl;
    return 0;
}
```

Expected Output: 100C = 212F

5.3 Constructor with Initialization List

An initialisation list sets member data before the constructor body executes.

ClassName(params) : member1(val1), member2(val2) { }

```
#include <iostream>
#include <cmath>
using namespace std;

class Vector2D {
public:
    const double x;   // const member: MUST use init list
    const double y;

    Vector2D(double a, double b) : x(a), y(b) { }  // init list

    double magnitude() { return sqrt(x*x + y*y); }
};
```

```
int main() {
    Vector2D v(3.0, 4.0);
    cout << "Magnitude: " << v.magnitude() << endl;
    return 0;
}
```

Expected Output: Magnitude: 5

Important: Always prefer initialisation lists over assignment in the constructor body. They are more efficient.

5.4 Constructor Overloading

Just like regular functions, constructors can be overloaded. The compiler selects the correct version based on the arguments supplied at object creation.

```
#include <iostream>
using namespace std;

class Box {
public:
    double l, w, h;

    Box() : l(1), w(1), h(1) {}              // default
    Box(double side) : l(side), w(side), h(side) {}  // cube
    Box(double a, double b, double c) : l(a), w(b), h(c) {} // full

    double volume() { return l * w * h; }
    void   display() { cout << l << "x" << w << "x" << h
                            << " vol=" << volume() << endl; }
};

int main() {
    Box b1;          b1.display();  // 1x1x1
    Box b2(4.0);     b2.display();  // 4x4x4
    Box b3(2,3,5);   b3.display();  // 2x3x5
```

```
    return 0;
}
```

Expected Output: 1x1x1 vol=1 4x4x4 vol=64 2x3x5 vol=30

Practice Tasks

Task 4: Create a class Person with members name (string) and age (int). Write three overloaded constructors: a default constructor (name="Unknown", age=0), one that accepts only name, and one that accepts both name and age. Create one object with each constructor and print the details.

6. Structs as Class Member Data

A struct in C++ is nearly identical to a class (the only difference is that members are public by default). Structs are often used for simple data groupings, and they can be embedded inside a class as member data to organise related fields.

```
#include <iostream>
using namespace std;

struct Date {        // struct for date info
    int day;
    int month;
    int year;
};

class Employee {
public:
    string name;
    Date   hireDate;  // struct as member data
    double salary;

    Employee(string n, Date d, double s) : name(n), hireDate(d), salary(s) {}

    void display() {
        cout << name << " (hired " << hireDate.day << "/"
```

```
            << hireDate.month << "/" << hireDate.year
            << ") Salary: " << salary << endl;
    }
};

int main() {
    Date d = {15, 3, 2020};
    Employee e("Zara", d, 75000.0);
    e.display();
    return 0;
}
```

Expected Output: Zara (hired 15/3/2020) Salary: 75000

Practice Tasks

Task 5: Create a struct Address with fields street (string), city (string), and postalCode (int). Create a class Hospital with members name (string) and location (Address). Write a constructor and a display() function. Create one Hospital object and print its details.

7. Returning Objects from Functions

Functions can return objects just like any other data type. The returned object is typically a new object computed from inputs. If the object is large, consider returning by reference or using move semantics, though for learning purposes, returning by value is standard.

```
#include <iostream>
using namespace std;

class Fraction {
public:
    int numerator;
    int denominator;

    Fraction(int n, int d) : numerator(n), denominator(d) {}

    void display() {
```

```
        cout << numerator << "/" << denominator << endl;
    }
};

// Returns a new Fraction object
Fraction add(Fraction a, Fraction b) {
    int n = a.numerator   * b.denominator
          + b.numerator   * a.denominator;
    int d = a.denominator * b.denominator;
    return Fraction(n, d);   // return object
}

int main() {
    Fraction f1(1, 3);
    Fraction f2(1, 6);
    Fraction result = add(f1, f2);
    result.display();
    return 0;
}
```

Expected Output: 9/18

Key Points

- Specify the class name as the return type in the function signature.
- The returned object is a copy; the local object inside the function is destroyed.

Practice Tasks

Task 6: Write a function largerBox(Box a, Box b) that takes two Box objects (use the class from Section 5.4) and returns the Box with the larger volume. Print the dimensions of the returned box in main().

8. Static Data Members

A static data member belongs to the class itself, not to any individual object. There is only one copy of a static member, shared by all objects of the class. Static members must be defined outside the class body (in addition to being declared inside).

static data_type memberName; // declaration inside class data_type
ClassName::memberName = value; // definition outside

```
#include <iostream>
using namespace std;

class Counter {
public:
    static int count;   // declaration: shared by all objects
    string label;

    Counter(string l) : label(l) {
        count++;        // increments the SHARED counter
    }

    static void showCount() {
        cout << "Total objects created: " << count << endl;
    }
};

int Counter::count = 0;  // definition and initialisation

int main() {
    Counter a("first");
    Counter b("second");
    Counter c("third");
    Counter::showCount();
    return 0;
}
```

Expected Output: Total objects created: 3

Key Points

- Static members exist even before any object is created.
- Access via ClassName::member or through any object (but the class name is clearer).

- Static member functions can only access static data.

Practice Tasks

Task 7: Create a class Sensor with a static int totalSensors and an int id. Each time a Sensor object is created, assign it the next available id (starting from 1) using totalSensors. Create 4 sensors and print each sensor's id and the total count.

9. Const Member Functions and Const Objects

9.1 Const Member Functions

A const member function promises not to modify any member data. Declare it by placing the keyword const after the parameter list. Only const member functions can be called on a const object.

```
return_type functionName(params) const;
```

```
#include <iostream>
using namespace std;

class Circle {
private:
    double radius;
public:
    Circle(double r) : radius(r) {}

    double area()   const { return 3.14159 * radius * radius; }
    double getRadius() const { return radius; }

    // Non-const: allowed to modify
    void setRadius(double r) { radius = r; }
};

int main() {
    Circle c(5.0);
    cout << "Area:   " << c.area()      << endl;
    cout << "Radius: " << c.getRadius() << endl;
    return 0;
}
```

Expected Output: Area: 78.5398 Radius: 5

9.2 Const Objects

A const object cannot have its member data changed after construction. The compiler enforces that only const member functions are called on it. This is useful for objects that should never be modified — like mathematical constants or configuration data.

```
#include <iostream>
using namespace std;

class Config {
private:
    int maxUsers;
    string appName;
public:
    Config(int m, string n) : maxUsers(m), appName(n) {}

    int    getMaxUsers() const { return maxUsers; }
    string getAppName()  const { return appName;  }
    // void setMax(int m) { maxUsers = m; }  // non-const: not callable on const obj
};

int main() {
    const Config cfg(100, "MyApp");  // const object
    cout << cfg.getAppName()  << endl;
    cout << cfg.getMaxUsers() << endl;
    // cfg.setMax(200);  // ERROR: cannot call non-const function
    return 0;
}
```

Expected Output: MyApp 100

Key Points

- A const function that accidentally tries to modify member data causes a compile error.
- Use const objects for read-only configuration, fixed constants, or function parameters that should not change.

Practice Tasks

Task 8: Create a class MathConstants with two private const doubles: pi (3.14159) and e (2.71828). Initialize them using a constructor initializer list. Provide const getter functions getPi() and getE(). In main(), create a const MathConstants object and use the getters to print both values.

10. Data Hiding: Public and Private Access (Encapsulation)

Encapsulation is one of the four pillars of Object-Oriented Programming. It means bundling data and the functions that operate on it inside a class, and controlling which parts are accessible from outside. This is achieved using access specifiers.

10.1 Access Specifiers

C++ provides three access specifiers. In this lab we focus on the two most important:

- public — members are accessible from anywhere in the program.
- private — members are accessible only from within the class itself (default for class).
- protected — used with inheritance

```
class ClassName
{
private:     // hidden from outside — internal state
data_type memberVariable;
public:     // accessible from outside — controlled interface
return_type memberFunction();
};
```

Important: A class member that is not preceded by any specifier is private by default. Making data private and exposing it only through public functions is the essence of data hiding.

10.2 Why Hide Data?

If member data is public, any part of the program can change it freely — like setting a bank balance to negative. Making data private forces all changes to go through member functions, where you can validate the new value before accepting it.

```
#include <iostream>
using namespace std;

class Student {
private:                    // hidden — cannot be accessed directly
    string name;
    int    age;
    double gpa;
```

```
public:                    // controlled interface
    // Setters with validation
    void setName(string n) { name = n; }

    void setAge(int a) {
        if (a >= 0 && a <= 120)  age = a;
        else cout << "Invalid age!" << endl;
    }
    void setGPA(double g) {
        if (g >= 0.0 && g <= 4.0)  gpa = g;
        else cout << "Invalid GPA!" << endl;
    }
    // Getters (const: read-only access)
    string getName() const { return name; }
    int    getAge()  const { return age;  }
    double getGPA()  const { return gpa;  }
    void display() const {
        cout << name << " | Age: " << age << " | GPA: " << gpa << endl;
    }
};
int main() {
    Student s;
    s.setName("Fatima");
    s.setAge(20);
    s.setGPA(3.7);
    s.display();
    s.setAge(-5);       // rejected by setter
    s.setGPA(9.9);      // rejected by setter
    s.display();        // data unchanged
    // s.age = -5;      // ERROR: 'age' is private
    return 0;
}
```

Expected Output: Fatima | Age: 20 | GPA: 3.7 Invalid age! Invalid GPA! Fatima | Age: 20 | GPA: 3.7

Key Points

- private data can only be read or changed by the class's own member functions.
- public setter functions act as gatekeepers: they validate new values before storing them.
- public getter functions provide read-only access without exposing the underlying variable.
- This pattern (private data + public getters/setters) is called the accessor/mutator pattern and is standard C++ style.

10.3 The Difference Between struct and class

In C++, a struct and a class are almost identical. The only difference is the default access level: struct members are public by default, while class members are private by default. Structs are typically used for plain data groupings; classes are used when encapsulation matters.

```
struct Point {        // members are public by default
    int x;
    int y;
};

class SafePoint {     // members are private by default
    int x;
    int y;
public:
    SafePoint(int a, int b) : x(a), y(b) {}
    int getX() const { return x; }
    int getY() const { return y; }
};

int main() {
    Point p;  p.x = 3;          // OK: public by default
    SafePoint sp(3, 4);
    // sp.x = 3;                // ERROR: private
    cout << sp.getX() << endl;   // OK: via getter
}
```

Expected Output: 3

Practice Tasks

Task 9: Create a class BankAccount with private members accountNumber (string), balance (double), and pin (int). Write a constructor that sets all three. Provide public functions: deposit(double amount), withdraw(double amount, int enteredPin) — only allow withdrawal if the pin matches and balance is sufficient — and getBalance() const. Test all operations in main().

Task 10: Create a class Temperature with a private double celsius. Write a constructor, a setter setCelsius(double) that rejects values below -273.15 (absolute zero), and const getters getCelsius(), getFahrenheit(), and getKelvin(). In main(), create a Temperature object, try setting an invalid value, then print all three scales.

11. Challenge Exercise

Library Book Management System

Build a mini library system that combines all concepts from this lab, including encapsulation through private data and public controlled interfaces:

```
// Requirements:
// 1. Define a struct PublisherInfo with public fields: name (string) and year (int).
//
// 2. Define a class Book with:
//
//   PRIVATE (data hiding / encapsulation):
//     title      (string)
//     author     (string)
//     price      (double)         -- must be > 0
//     publisher  (PublisherInfo)   -- struct as member data
//     available  (bool)           -- true = on shelf
//     static int totalBooks        -- shared count of all books created
//
//   PUBLIC (controlled interface):
//     Overloaded constructors:
//       Book()                                  // default
//       Book(string t, string a, double p)          // no publisher
//       Book(string t, string a, double p,
```

```
//            PublisherInfo pub)                    // full
//
//     Setter with validation:
//       setPrice(double p)   -- print error and reject if p <= 0
//
//     Const getters:
//       getTitle(), getAuthor(), getPrice(), isAvailable()
//
//     Other public functions:
//       checkOut()         -- mark unavailable; print error if already out
//       returnBook()       -- mark available again
//       display() const    -- print all book details
//       static showTotal() -- print totalBooks
//
// 3. Write a standalone function:
//    Book cheapest(Book a, Book b)
//    -- takes two Book objects by value
//    -- uses const getters to compare prices
//    -- returns the cheaper Book object
//
// 4. In main():
//    a. Create 3 books using different constructors
//    b. Try setting an invalid price (0 or negative) -- observe rejection
//    c. Display all 3 books using display()
//    d. Check out one book; try checking it out again (error message)
//    e. Return the book; check it out again (should now succeed)
//    f. Call cheapest() and display the returned book
//    g. Print total books via the static function
```

Sample Output :

```
Invalid price! Price must be greater than 0.


--- All Books ---
Title: Clean Code      | Author: Robert Martin | Price: 2500 | Publisher: Pearson (2008)
| Available: Yes
```

```
Title: The Pragmatic   | Author: Hunt & Thomas | Price: 3200 | Publisher: Addison (1999) | Available: Yes
Title: Design Patterns | Author: GoF           | Price: 4000 | No publisher info        | Available: Yes

'Clean Code' checked out successfully.
Error: 'Clean Code' is not available.
'Clean Code' has been returned.
'Clean Code' checked out successfully.

Cheapest book:
Title: Clean Code | Author: Robert Martin | Price: 2500 | Available: No

Total books in system: 3
```

12. Concept Summary

Concept	Key Syntax / Feature	When to Use
Class Definition	class Name { public: ... };	Model a real-world entity with data + behaviour
Member Data	int x; string name;	Store object state inside the class
Member Function	void display() { }	Define behaviour that operates on member data
Scope Resolution (::)	void Name::func() { }	Define member functions outside the class body
Creating Objects	ClassName obj;	Allocate memory for a class instance
Objects as Arguments	void f(ClassName obj)	Pass objects to functions (by value or reference)
Constructor	Name(params) { ... }	Auto-initialise members when object is created
Init List	: m1(v1), m2(v2) { }	Initialise members before constructor body runs

Constructor Overloading	Name() / Name(int) / Name(int,int)	Support multiple creation patterns
Struct as Member	struct S { }; class C { S s; };	Group related fields into a named sub-object
Return Object	ClassName func() { return ClassName(...); }	Compute and hand back a new object from a function
Static Data Member	static int count; / int C::count = 0;	Share one variable across all objects
Const Member Function	int get() const { }	Guarantee no member data is modified
Const Object	const ClassName obj;	Prevent all modification after construction
private (Data Hiding)	private: int x;	Restrict direct access; protect internal state
public (Interface)	public: void setX(int);	Expose only validated, controlled operations
Encapsulation	private data + public getters/setters	Bundle data and behaviour; hide implementation details

Common Mistakes to Avoid

- Forgetting the semicolon after the closing brace of a class definition.
- Omitting the :: operator when defining a member function outside the class.
- Trying to modify member data inside a const function (compile error).
- Calling a non-const member function on a const object (compile error).
- Forgetting to define a static member outside the class body (linker error).
- Not using an initialisation list for const or reference members (compile error).
- Forgetting that each object has its own copy of non-static member data.
- Making all members public — this breaks encapsulation and removes data protection.
- Writing a setter without validation — defeats the purpose of data hiding.